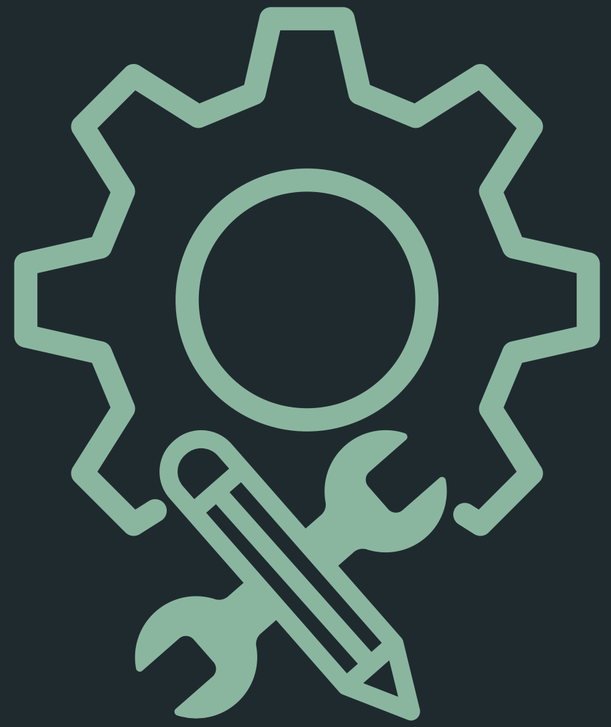
ShelfLife

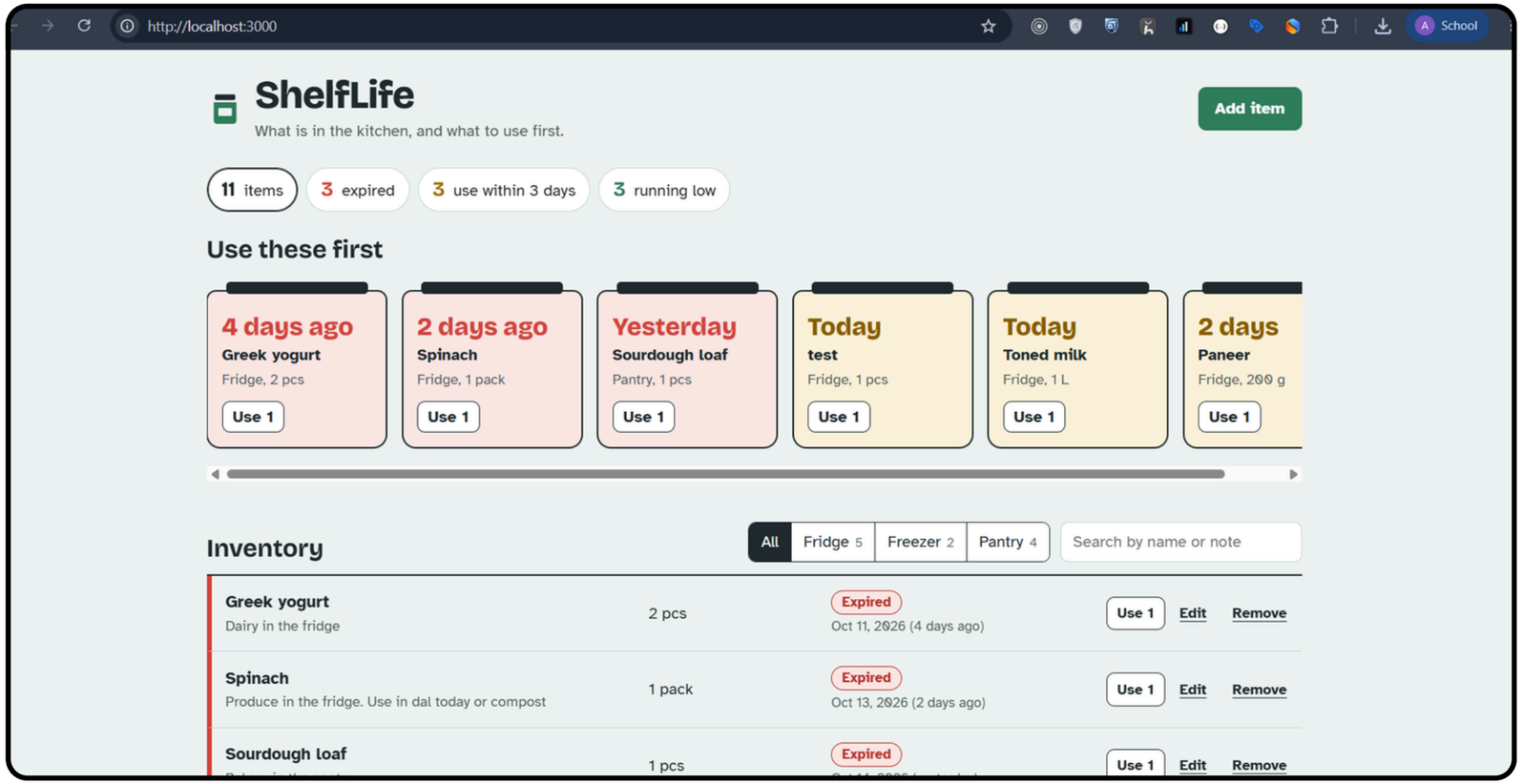
From a laptop to a monitored Kubernetes deployment: a DevSecOps capstone



Submitted By: Anushika Chauhan | Group-A | Roll Number: 10344

What ShelfLife does

Food goes off because nobody remembers what is at the back of the fridge.



ShelfLife running locally at http://localhost:3000 (Docker Compose)

- Record each item: where it is stored, how much is left, and its use-by date
- The "Use these first" shelf shows expired and soon-to-expire food first
- Counts and filters: expired, use within 3 days, running low
- "Use 1" updates the quantity; everything is saved in PostgreSQL

Tech stack

Application

React 19 + Vite, FastAPI, PostgreSQL 17, SQLAlchemy, Alembic

Quality

pytest (17 tests), Ruff lint, migration check on Postgres

Containers

Multi-stage Dockerfiles, non-root users, Docker Compose

CI/CD

GitHub Actions, GitHub Container Registry, commit-SHA tags

Security

Trivy gate on both images (HIGH/CRITICAL fails the build)

Cloud and IaC

Terraform, AWS VPC (2 public subnets), EKS 1.36, node group

Kubernetes

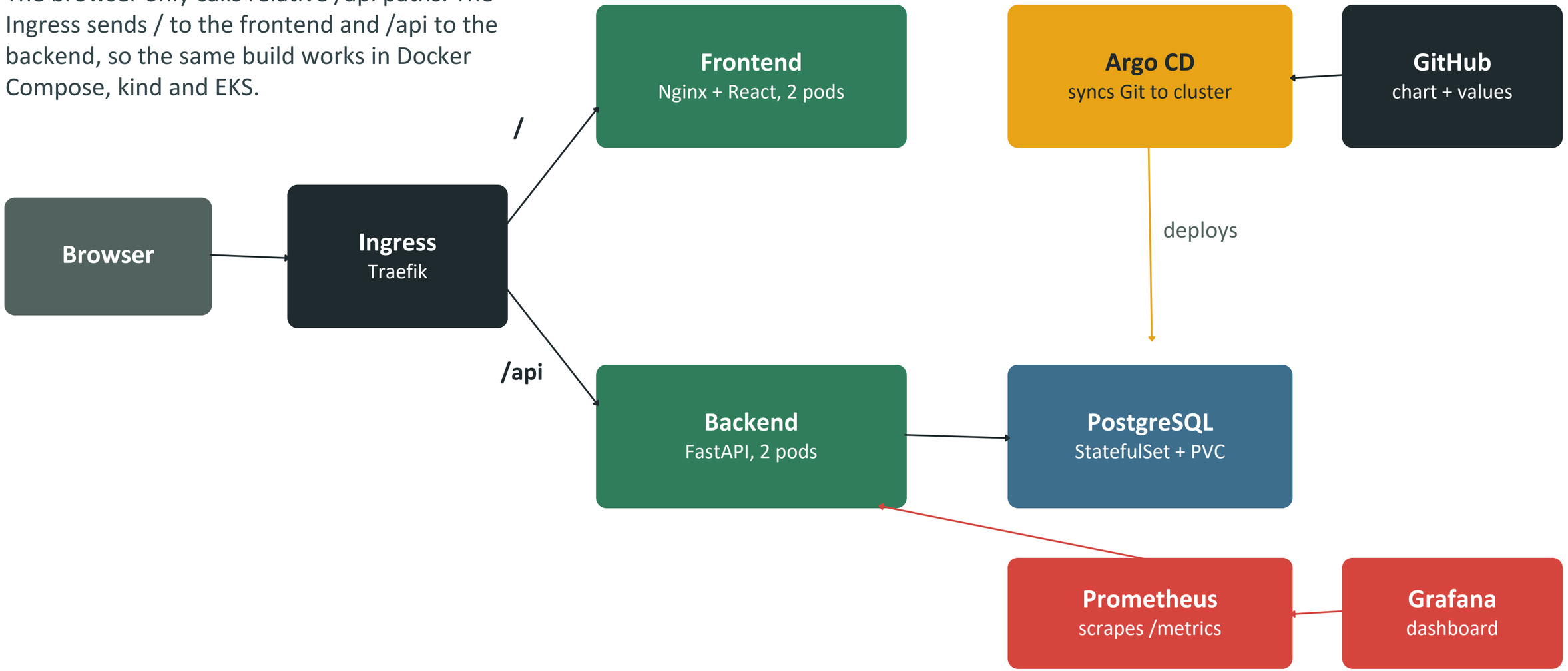
Helm chart, Traefik Ingress, kind for local and CI

Operations

Argo CD (GitOps), Prometheus, Grafana

Architecture

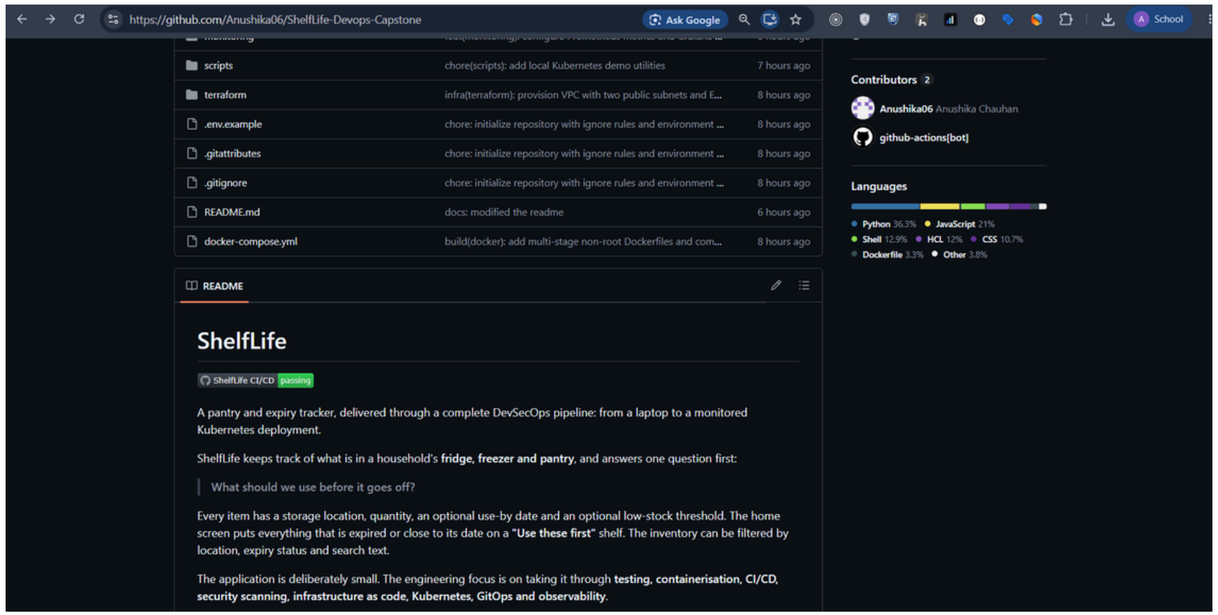
The browser only calls relative /api paths. The Ingress sends / to the frontend and /api to the backend, so the same build works in Docker Compose, kind and EKS.



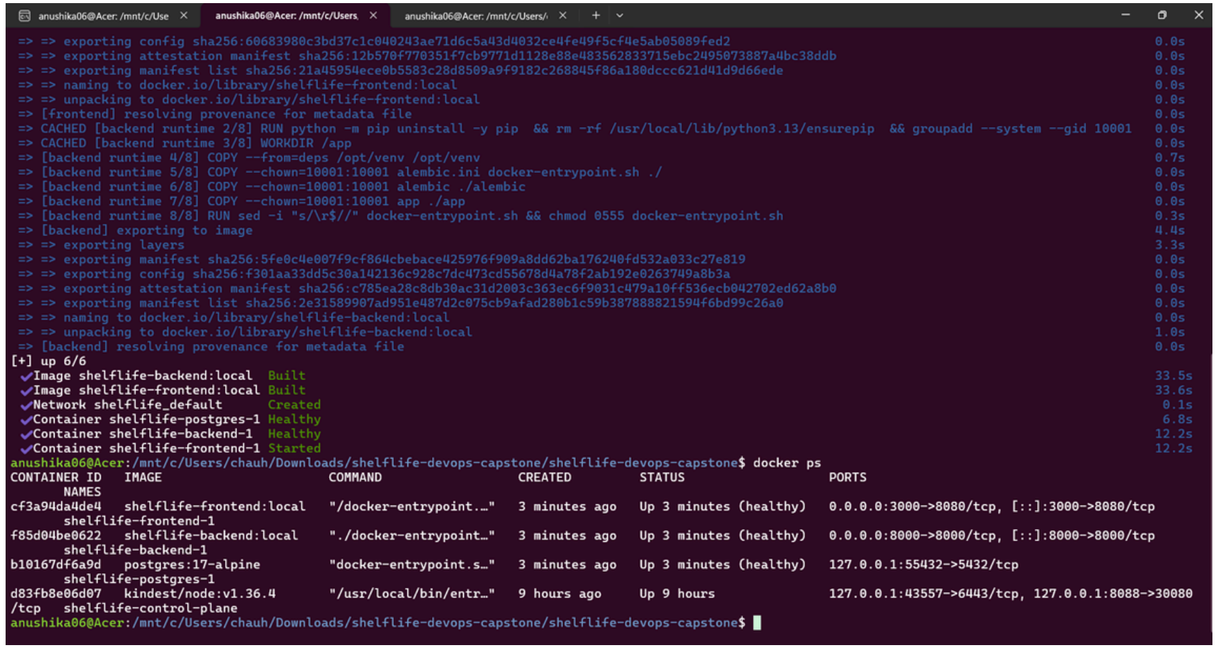
Codebase and local run

```
backend/      API, tests
frontend/     React UI
docker-compose.yml
.github/workflows/
terraform/    VPC + EKS
k8s/          namespaces
helm/         the chart
monitoring/   Prom + Grafana
argocd/       GitOps app
```

One command starts all three services:
docker compose up --build



GitHub repository: folders, README and the passing CI badge



docker compose up --build: postgres, backend and frontend healthy

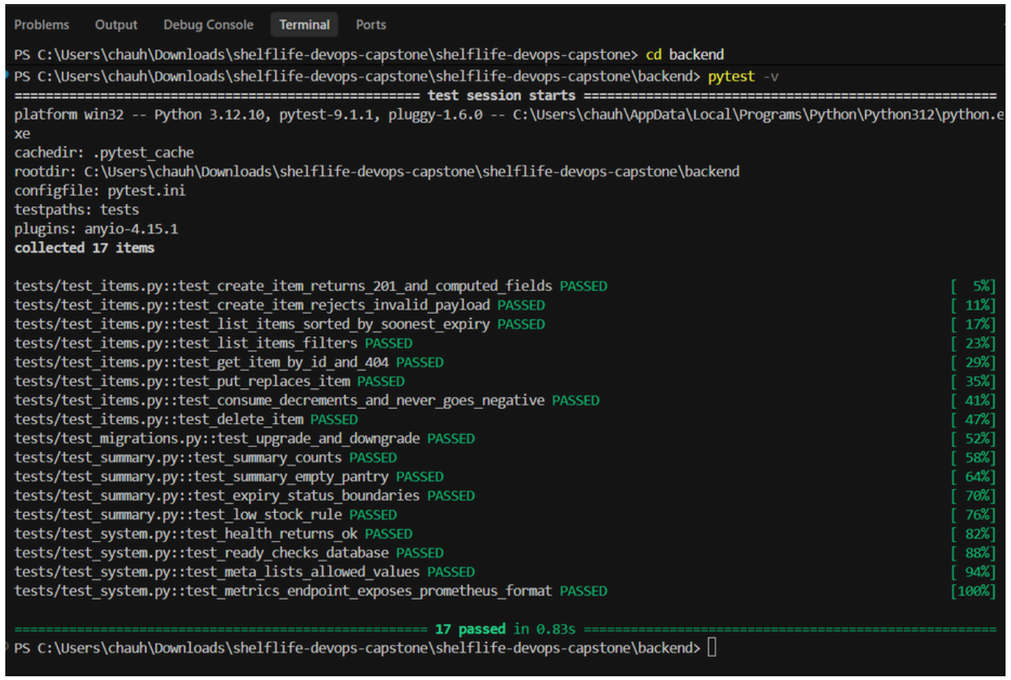
Testing and quality gates

17
pytest tests

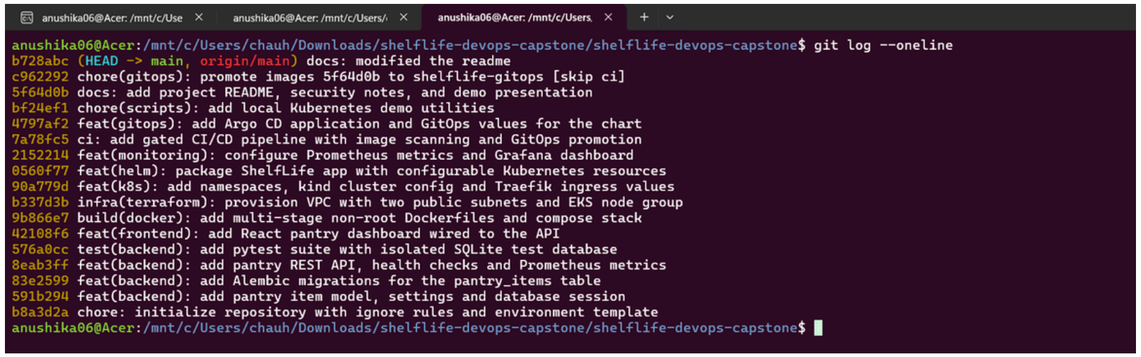
5
API areas covered

0
tests touching the real DB

- conftest.py swaps DATABASE_URL to a temporary SQLite file before the app loads
- Covers health, readiness, items CRUD, consume, summary, metrics and migrations
- CI also runs alembic upgrade, downgrade and upgrade on a real PostgreSQL 17
- Ruff lint and format check fail the build like a failing test



pytest -v: 17 passed



git log --oneline: meaningful, phased commits

CI/CD pipeline

Test
pytest, Ruff, Alembic

Build
frontend + 2 images

Scan
Trivy gate

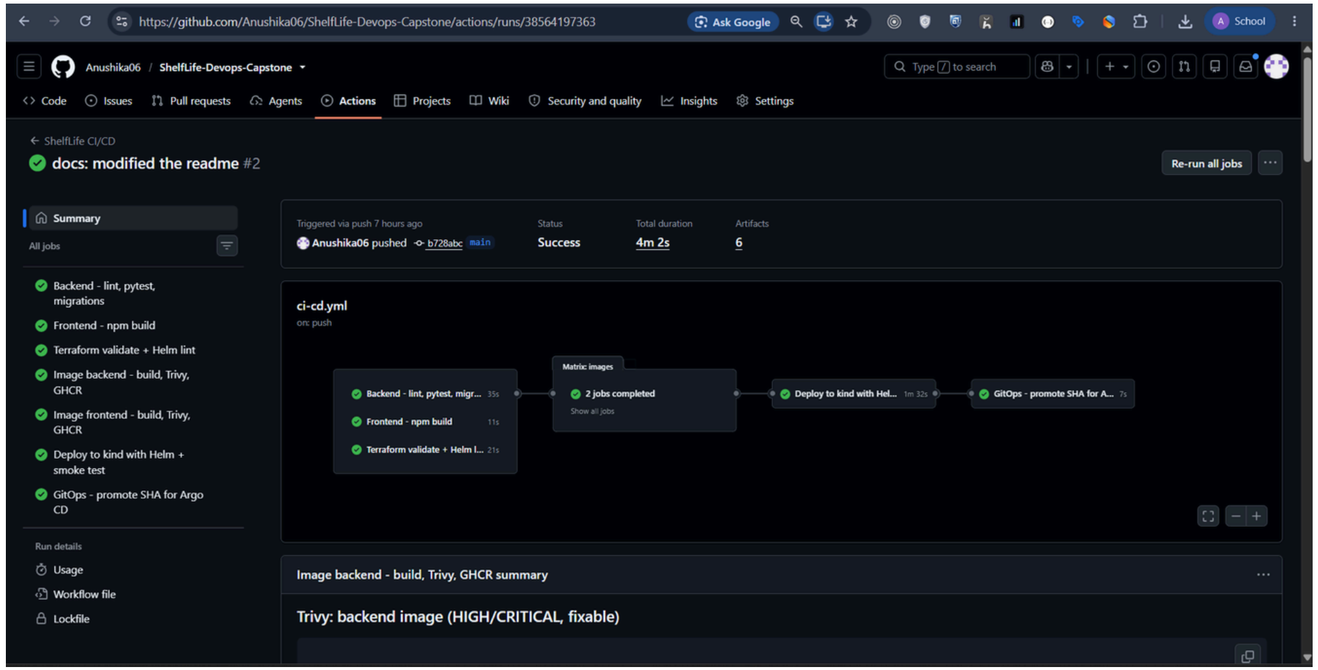
Publish
GHCR :git-sha

Deploy
kind + Helm + smoke

Promote
SHA into Git

Runs on every push to main. Each job needs the previous one, so a failed test or a HIGH/CRITICAL CVE stops the image from ever reaching the registry.

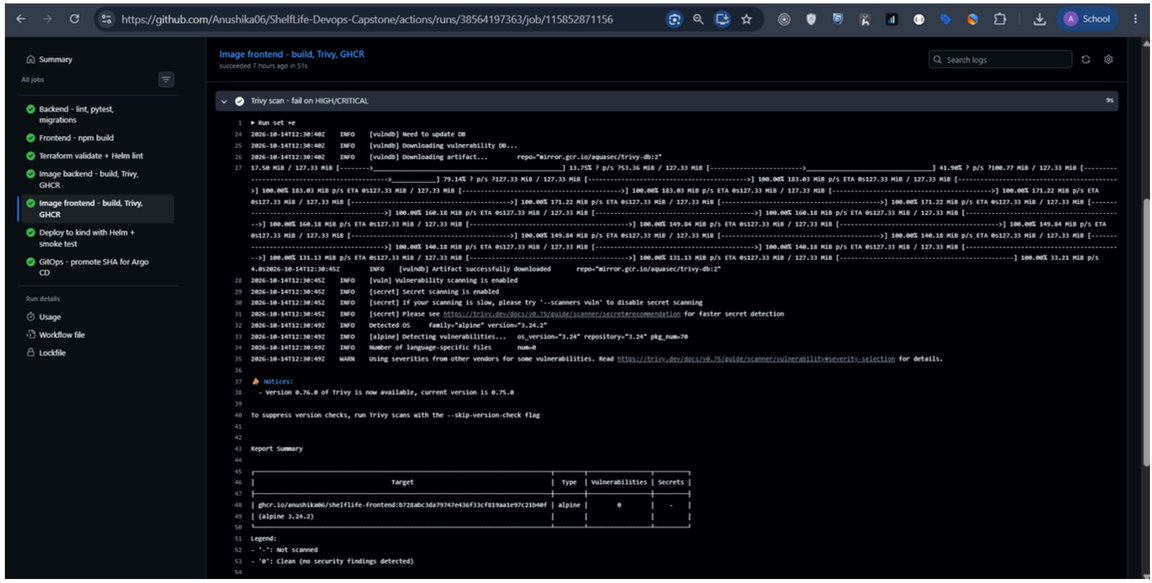
Images are tagged only with the commit SHA, never latest.



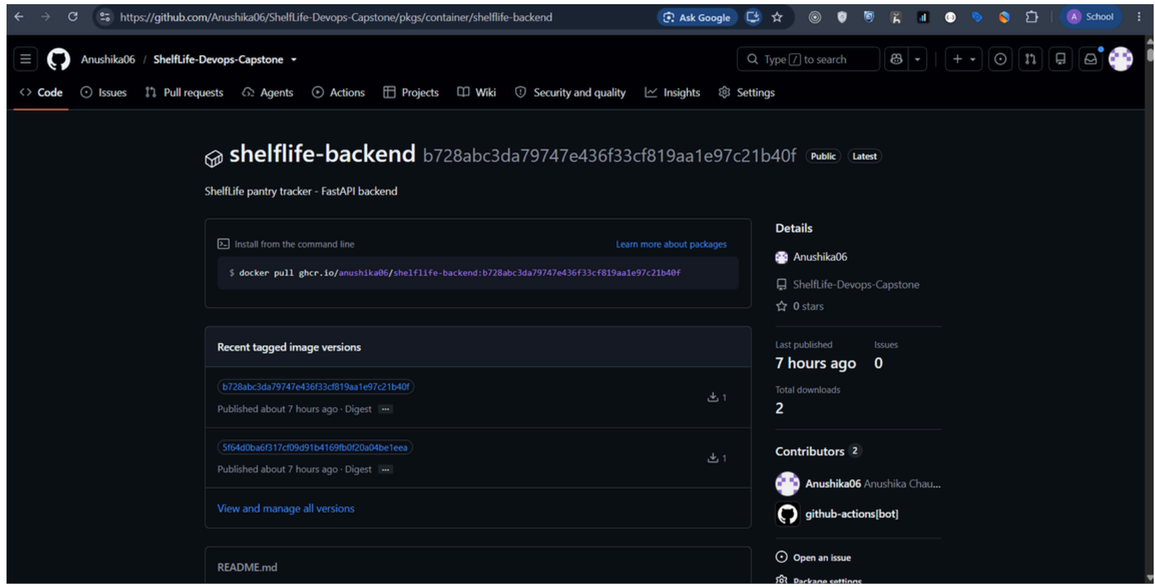
Run #2 on main (commit b728abc): all 7 jobs green, 4m 2s

DevSecOps: Trivy and GHCR

- trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 runs on both images before the push
- Pipeline run #2 (commit b728abc): no fixable HIGH/CRITICAL findings in either image, so both were pushed
- During development a local scan found 4 HIGH in pip's vendored libraries; pip is now removed from the runtime image
- GHCR login uses the built-in GITHUB_TOKEN; images are tagged with the full commit SHA

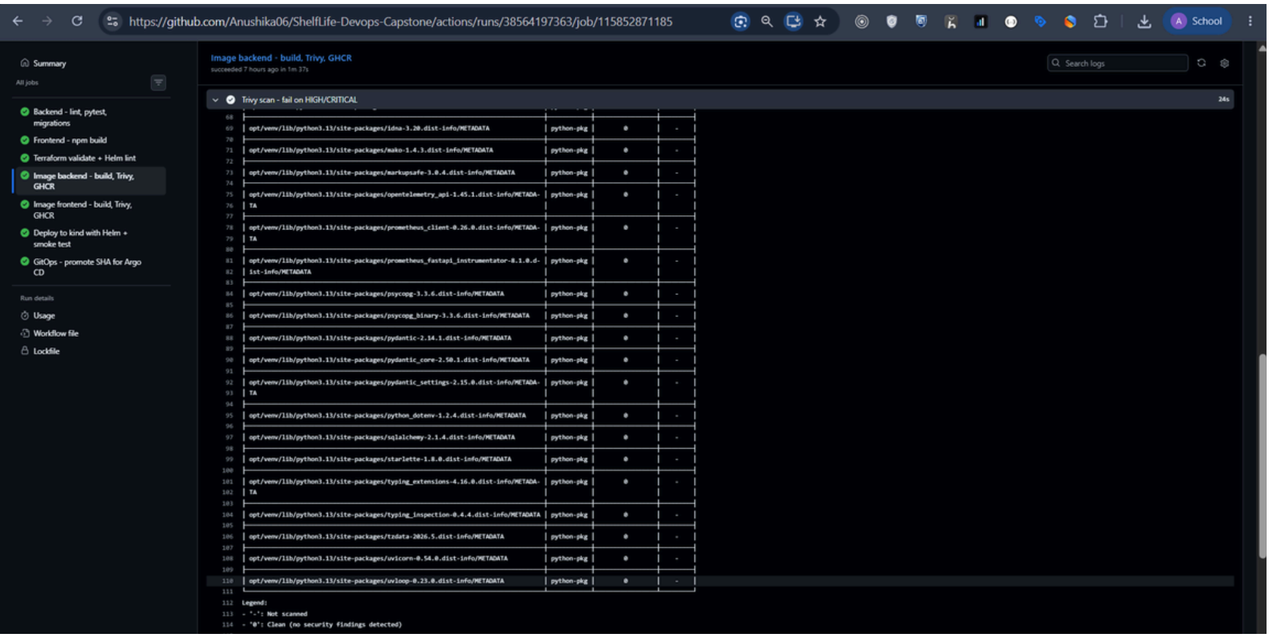


Trivy on the frontend image: alpine 3.24.2, 0 vulnerabilities

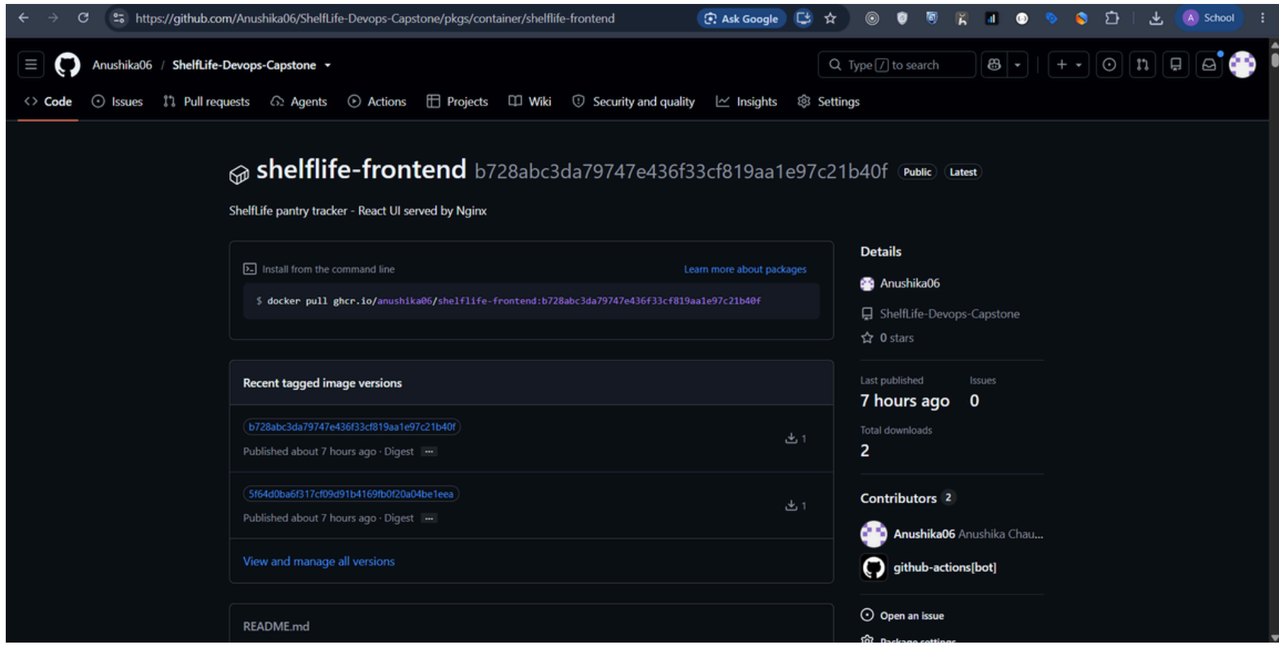


GHCR shelflife-backend: tags are commit SHAs

Security evidence for both images



Trivy on the backend image: every Python package 0 findings



GHCR shelflife-frontend: same SHA tag as the backend

Same pipeline run #2 (commit b728abc): neither image had a fixable HIGH or CRITICAL vulnerability, so the gate passed. Both packages carry the identical 40-character commit SHA tag, so every running container traces back to one commit.

Infrastructure as code on AWS

VPC 10.42.0.0/16 (ap-south-1)

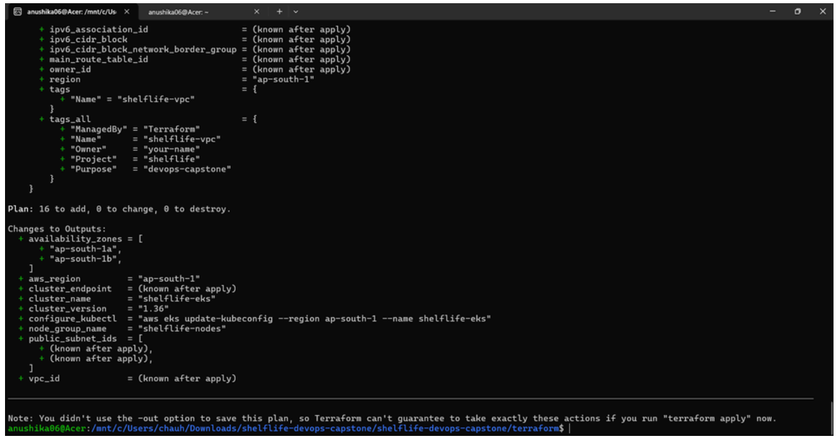
Public subnet A
AZ 1

Public subnet B
AZ 2

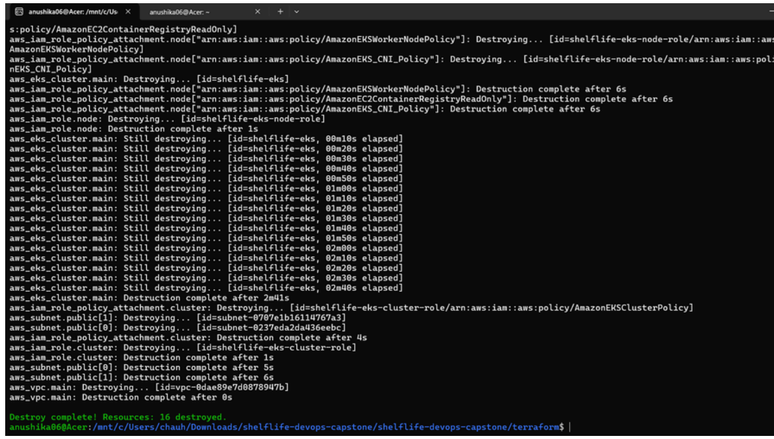
EKS 1.36 control plane
standard support only

Managed node group
2 x t3.small (evidence run)

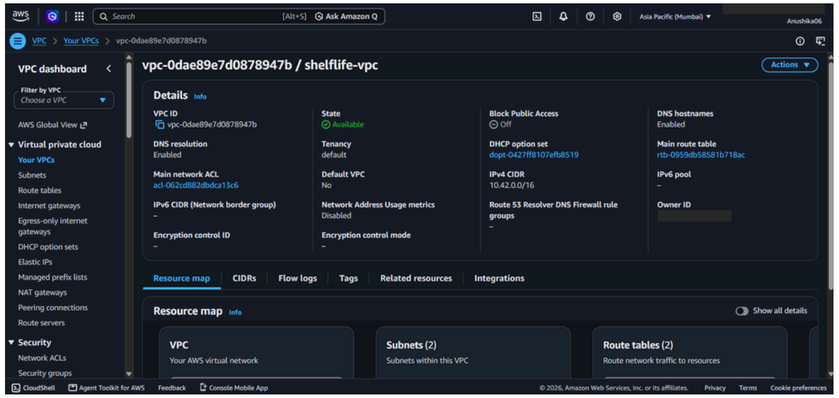
No NAT gateway, no control-plane logging, local state.
Destroyed right after the evidence was captured.



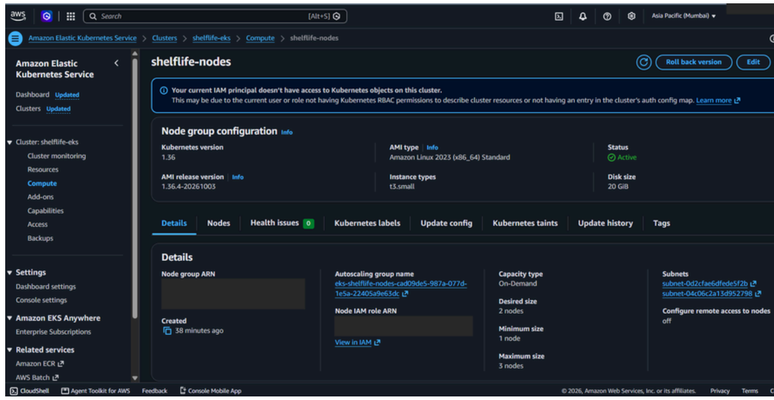
terraform plan: 16 to add



terraform destroy: 16 destroyed

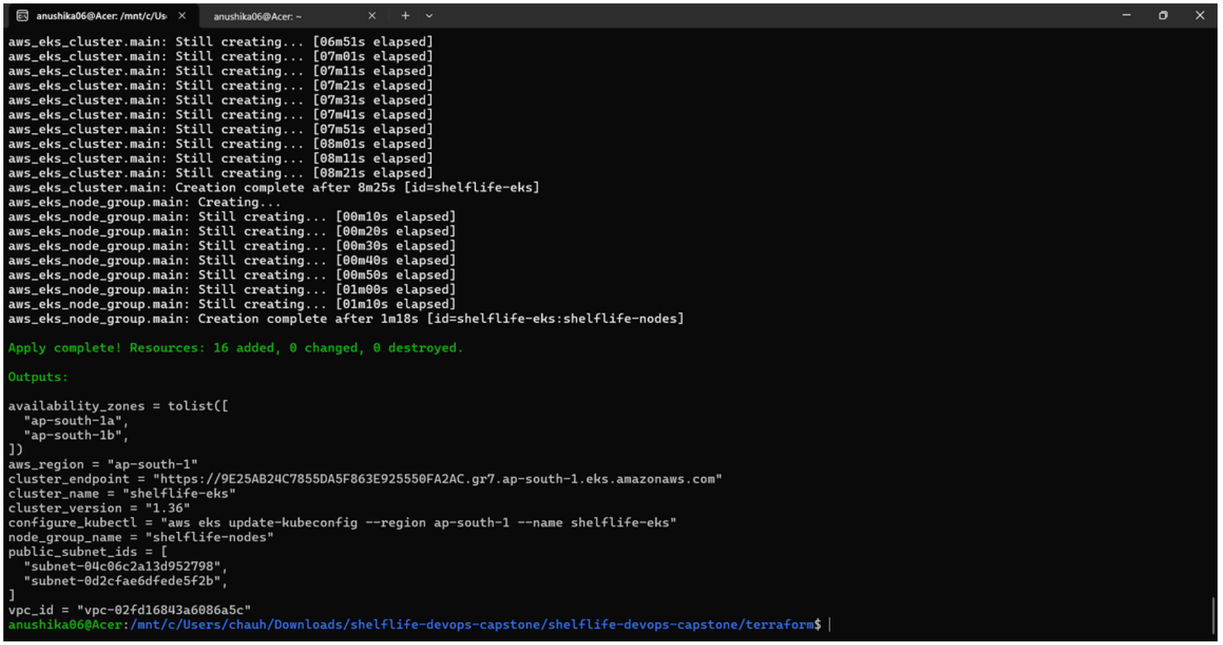


shelflife-vpc, 10.42.0.0/16

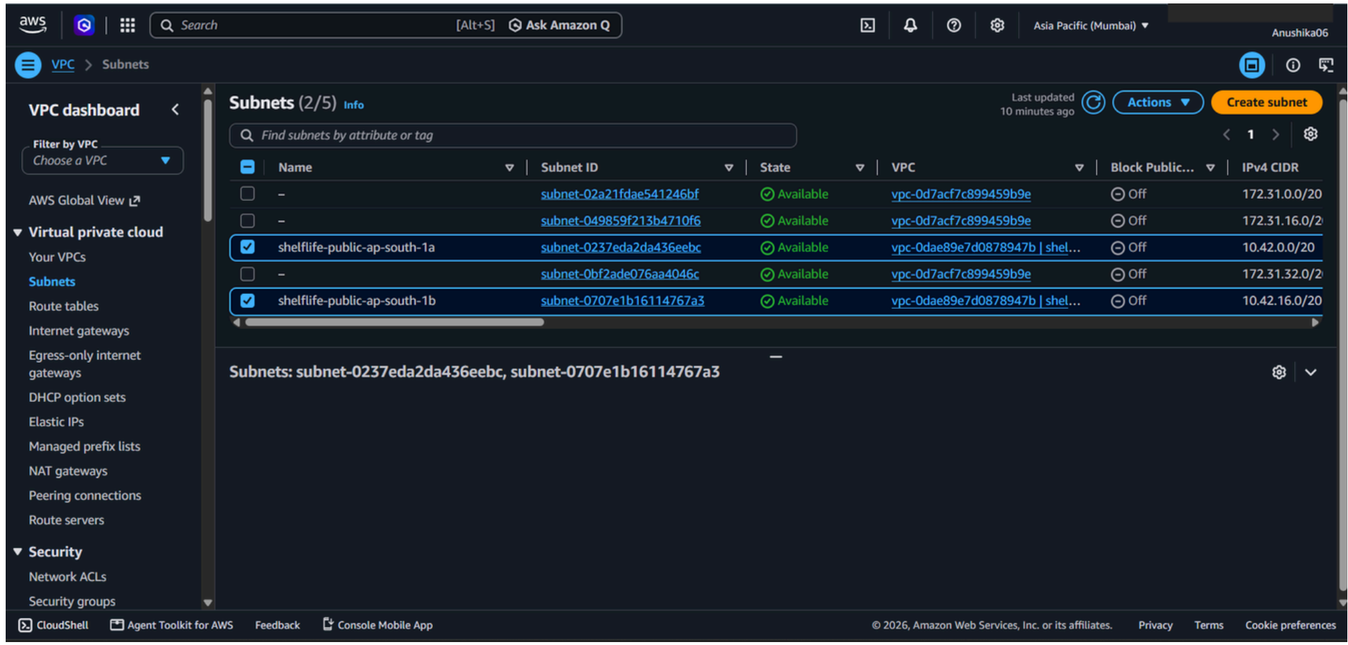


EKS node group shelflife-nodes Active

AWS evidence: apply, subnets, nodes, teardown



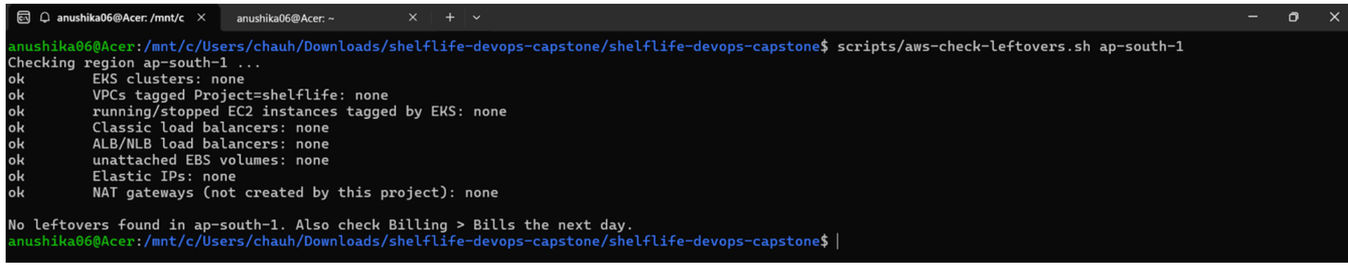
terraform apply: 16 added, outputs



Two public subnets in ap-south-1a and ap-south-1b



kubectl get nodes on EKS: 2 nodes Ready



scripts/aws-check-leftovers.sh: no leftovers after destroy

Kubernetes and Helm

```
anushika06@Acer:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$ kubectl get pods,svc,ingress -n shelflife
NAME                                    READY   STATUS    RESTARTS   AGE
pod/shelflife-backend-66cd4b4bbc-8s6gf   1/1     Running   0          4m44s
pod/shelflife-backend-66cd4b4bbc-b88hj   1/1     Running   0          4m44s
pod/shelflife-frontend-5899dd8885-rgg8v  1/1     Running   0          4m44s
pod/shelflife-frontend-5899dd8885-wlz5m  1/1     Running   0          4m44s
pod/shelflife-postgres-0                 1/1     Running   0          4m44s

NAME                        TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/shelflife-backend   ClusterIP   10.96.1.16      <none>        8000/TCP   4m44s
service/shelflife-frontend  ClusterIP   10.96.253.249   <none>        80/TCP     4m44s
service/shelflife-postgres  ClusterIP   10.96.251.189   <none>        5432/TCP   4m44s

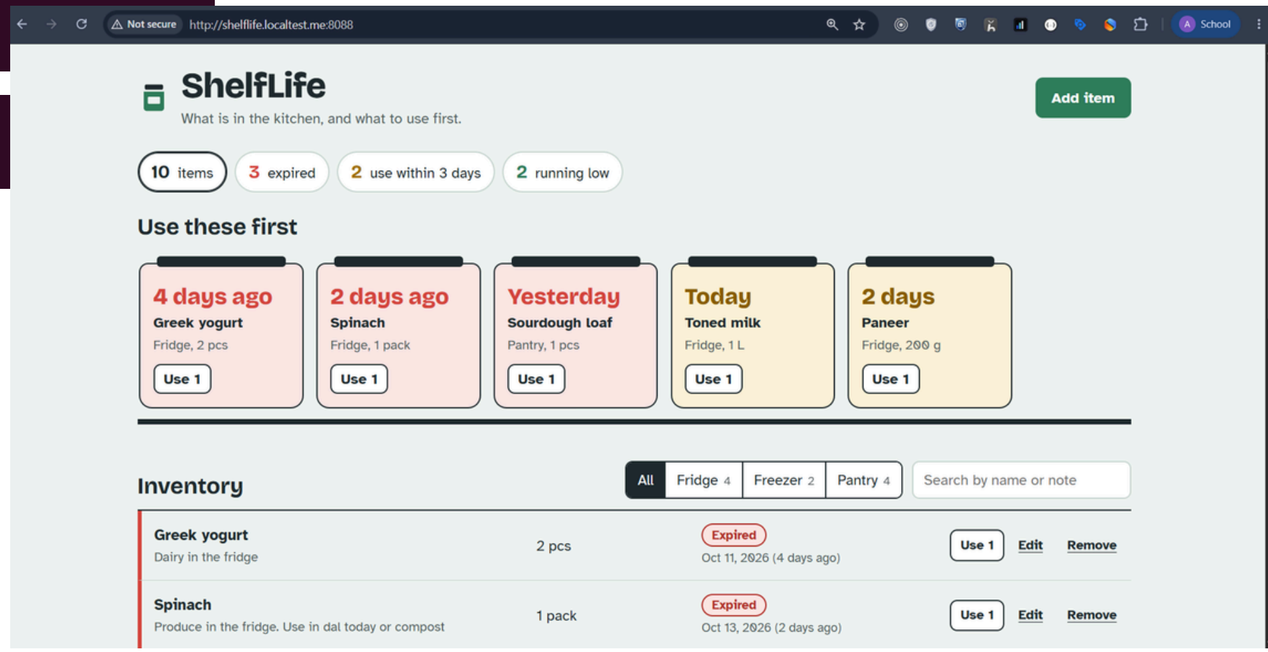
NAME                                    CLASS     HOSTS                   ADDRESS   PORTS   AGE
ingress.networking.k8s.io/shelflife     traefik   shelflife.localtest.me            80      4m44s
anushika06@Acer:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$
```

```
anushika06@Acer:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$ kubectl get svc -n shelflife
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
shelflife-backend   ClusterIP   10.96.1.16      <none>        8000/TCP   6m26s
shelflife-frontend  ClusterIP   10.96.253.249   <none>        80/TCP     6m26s
shelflife-postgres  ClusterIP   10.96.251.189   <none>        5432/TCP   6m26s
anushika06@Acer:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$
```

```
:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$ helm list -n shelflife
NAMESPACE       REVISION        UPDATED                                 STATUS          CHART           APP VERSION
shelflife       1               2026-10-14 19:50:54.455288159 +0000 UTC deployed        shelflife-0.1.0 1.0.0
:/mnt/c/Users/chauh/Downloads/shelflife-devops-capstone/shelflife-devops-capstone$
```

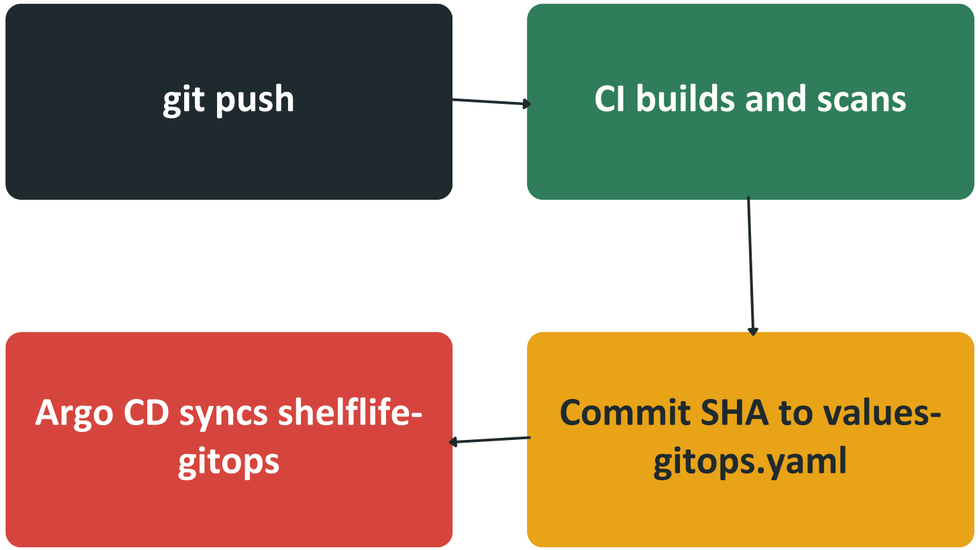
kind cluster: pods Running, ClusterIP services, Ingress, helm list (deployed)

- k8s/namespace.yaml creates the shelflife and shelflife-gitops namespaces
- helm upgrade --install deploys 2 backend + 2 frontend pods and PostgreSQL
- ClusterIP Services; Ingress routes / to frontend and /api to backend
- Probes, non-root, read-only filesystem, PodDisruptionBudgets, helm test
- Image tags are required values: the chart refuses to render without a SHA

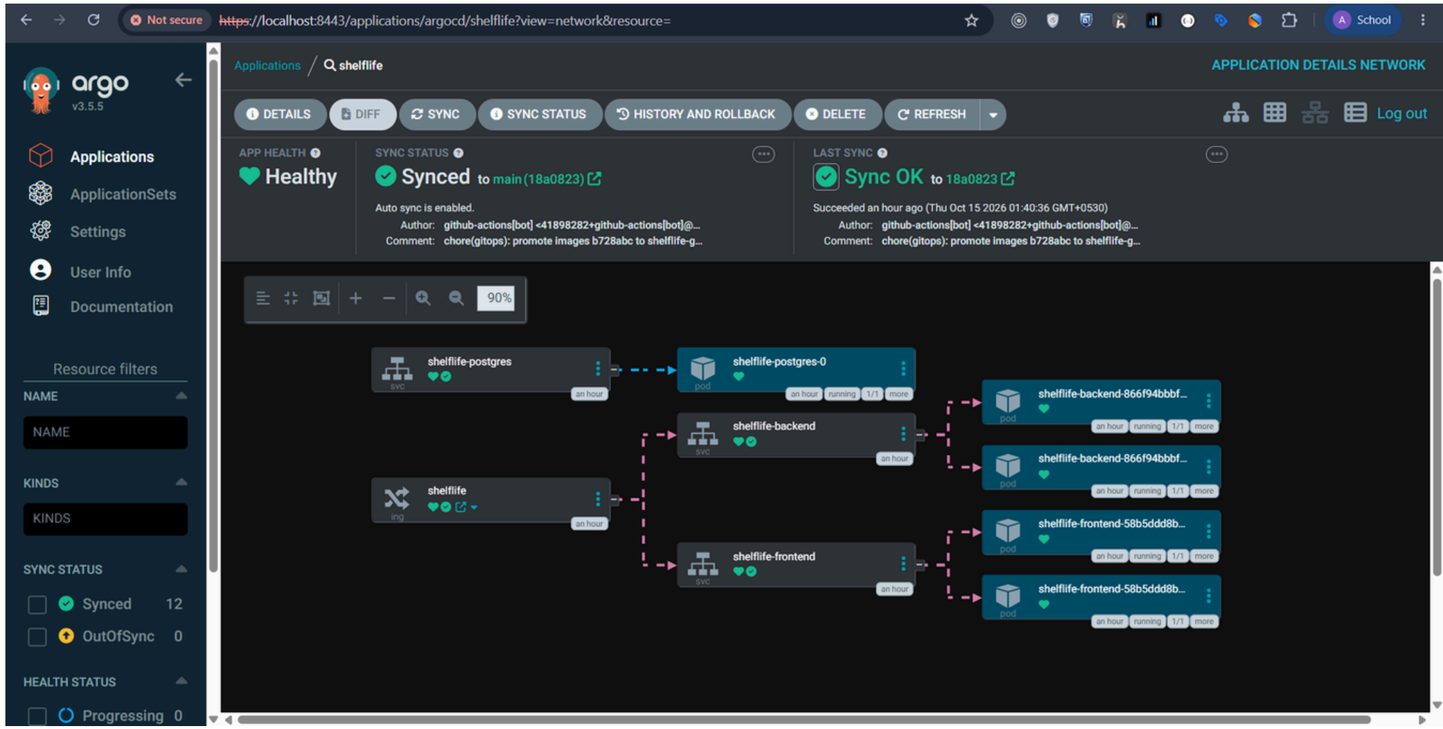


http://shelflife.localtest.me:8088 through the Traefik Ingress

GitOps with Argo CD



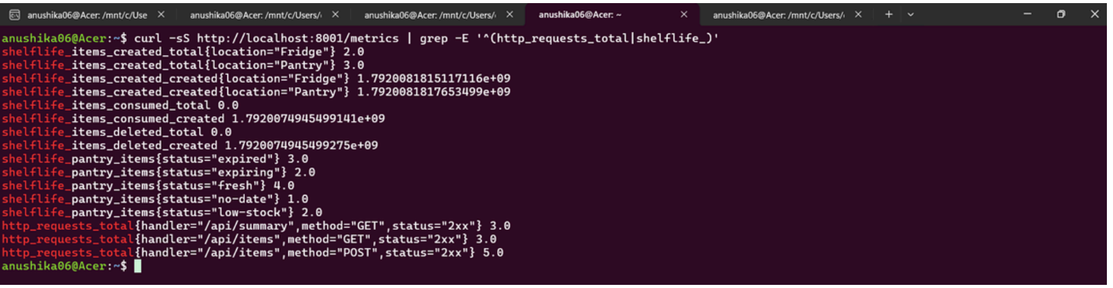
Git is the desired state. Automated sync with prune and self-heal: a manual kubectl change is reverted to what Git says.



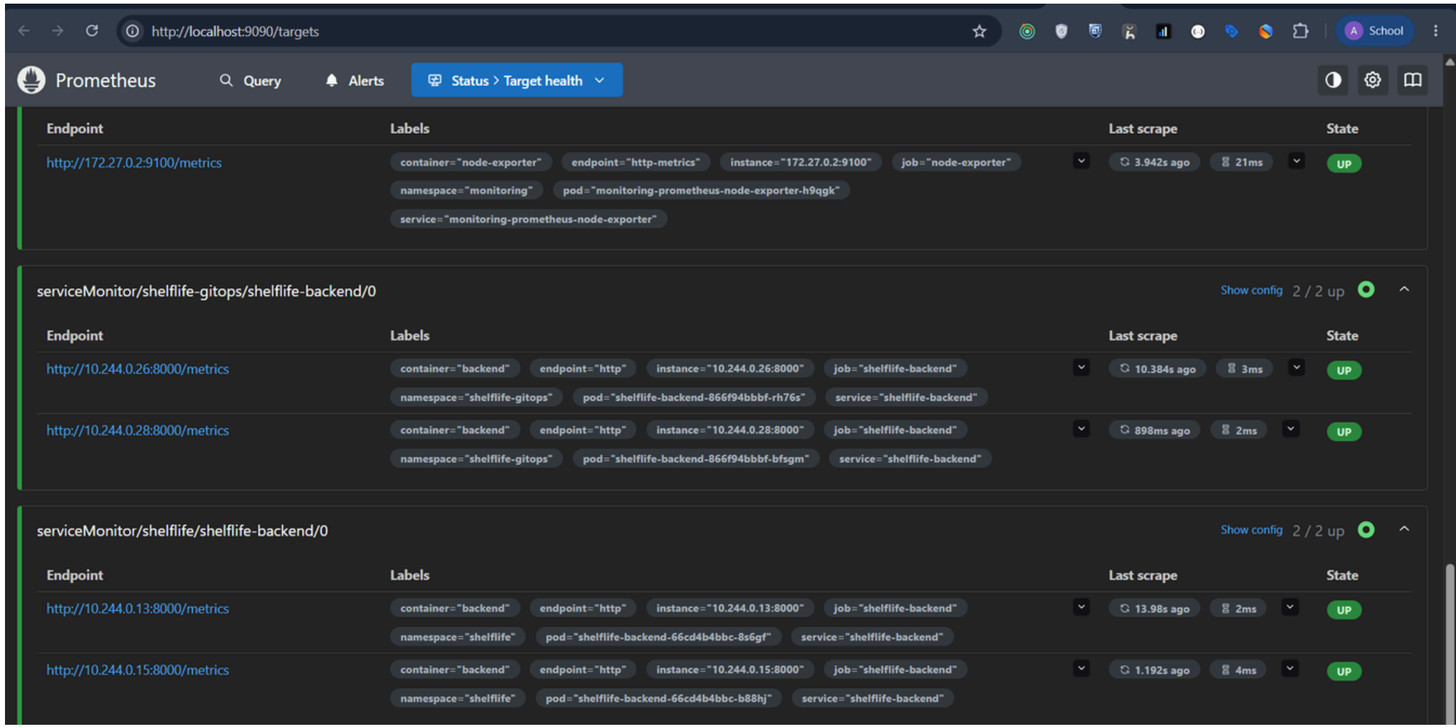
Argo CD: shelflife Healthy and Synced to main (18a0823), auto sync on

Observability

- Backend /metrics: request count, latency histogram, status class
- Business metrics: items added, used, removed, and items by status
- ServiceMonitor tells Prometheus to scrape both backend pods
- The ShelfLife API dashboard is provisioned automatically in Grafana



curl /metrics



Prometheus targets: shelflife and shelflife-gitops backends 2/2 UP

Grafana: ShelfLife API dashboard



2 backend pods UP, requests per second, p95 latency, requests by endpoint, pantry items by status

Alternatives and trade-offs

Decision	Alternative	Why this choice
Public subnets, no NAT	Private nodes + NAT gateway	Lower cost for a 1 to 2 day demo; production would use private subnets
PostgreSQL in the cluster	Amazon RDS	Free on kind and simple to demo; RDS adds backups and HA at extra cost
Traefik Ingress	ingress-nginx	ingress-nginx was archived in 2026 and gets no more fixes
kind for K8s demos	EKS for everything	No cloud cost; EKS is used for the Terraform proof
GITHUB_TOKEN for GHCR	Personal access token	Short-lived, scoped to the run, nothing to leak
Local Terraform state	S3 backend with locking	One person, short-lived; a team needs remote state

Results

Module	Evidence	Status at hand-in
M1 App + M4 Docker	E01, E02	Done: 3 containers healthy, UI at localhost:3000
M2 Testing	E04	Done: 17 of 17 tests passed (pytest -v)
M3 Git	E03, E05	Done: public repo, 20 meaningful commits
M5 CI/CD + M6 Trivy	E06, E07, E08	Done: run #2 green; SHA tags in GHCR; Trivy clean
M7 Terraform	E09 to E12	Done: plan 16 to add, applied in ap-south-1, destroyed
M8 Kubernetes + Helm	E13 to E16	Done on kind: 2+2 replicas, postgres, Ingress, helm test
M9 Observability	E18 to E20	Done: /metrics, targets 2/2 UP, Grafana populated
GitOps	E17	Done: Argo CD Synced + Healthy (18a0823)

All screenshots are in demo/evidence/ and were captured from real runs on 14-15 October 2026.

Key learnings

Gates, not just tools

A scan only matters when its result can stop a release.

Smaller images, fewer CVEs

Removing what the app never uses fixed every HIGH finding.

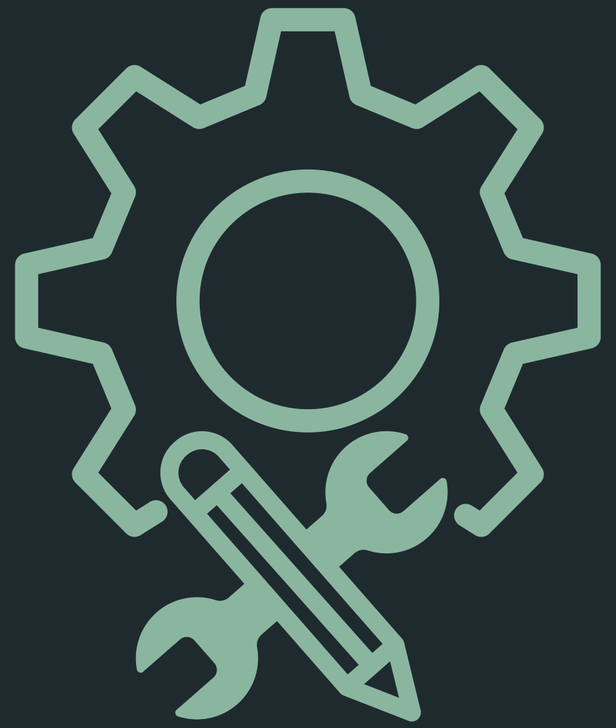
Cost is a design input

Support policy, NAT and load balancers change the AWS bill more than instance size.

Git as the source of truth

With Argo CD, the deployed version is a commit you can read and revert.

Thank you



Anushika Chauhan | Group-A

Repository: https://github.com/Anushika06/ShelfLife-Devops-Capstone